WHITEPAPER · VERSION 1.0 · OCTOBER 2026

inTrust

Milestone-based crowdfunding with on-chain escrow

Built on BOT Chain testnet (chain ID 968)

Prepared for the BOT Chain Africa Builder Challenge

Contact: Telegram @Oxsolmm

Contents

1. Abstract
2. The Problem
3. The inTrust Approach
4. Participants
5. Campaign Lifecycle
6. Governance and Payout Mathematics
7. Smart Contract Design
8. Application Layer
9. Deployment on BOT Chain
10. Security Considerations and Limitations
11. Roadmap
12. Conclusion

1. Abstract

inTrust is a decentralized crowdfunding protocol for BOT Chain in which donated funds are held in on-chain escrow and released to a creator one milestone at a time. A milestone is paid out only after backers, voting with weight proportional to the amount they gave, approve it by more than half of all funds raised. If a campaign misses its goal by the deadline, every backer can withdraw their contribution in full. The protocol has no administrator, no platform fee and no upgrade path: the rules that govern a campaign are the rules written in the contract at the moment it is created.

This paper explains the problem inTrust addresses, how a campaign moves from creation to completion, how the smart contract is structured, the trust assumptions that remain, and the improvements planned for later versions.

2. The Problem

Crowdfunding works when strangers are willing to back an idea they cannot yet inspect. Traditional platforms rely on the operator to hold the money, decide what counts as progress and arbitrate disputes. That model has three weaknesses that matter especially in emerging ecosystems.

- **All-or-nothing trust.** Backers send money and hope. Once funds leave their wallet there is no mechanism that ties the creator's payout to evidence of work.
- **Opaque custody.** Funds sit with a platform or the creator, and backers cannot independently verify balances, terms or payout history.
- **Weak recourse.** When a project stalls, backers depend on goodwill or on a platform's policy to recover anything, and cross-border backers have little leverage.

Builders across Africa feel this acutely. Local founders struggle to raise from distant backers who have no way to verify delivery, while backers have been burned often enough to hesitate. A funding primitive that makes delivery the condition for payment lowers the cost of trust for both sides.

3. The inTrust Approach

inTrust moves the three jobs that platforms normally perform (custody, rule enforcement and refunds) into a single immutable smart contract. Three design choices define it.

- **Escrow by default.** Every donation is held by the contract. A creator can never withdraw money that backers have not approved.
- **Milestones as the unit of payment.** A campaign is a goal, a deadline and an ordered list of milestones, each with a percentage of the funds. The percentages must total exactly 100.
- **Backers are the verifiers.** Approval is a vote weighted by contribution. No oracle, administrator or arbiter decides whether work was delivered; the people who funded it do.

4. Participants

Role	What they can do	What they cannot do
Creator	Create a campaign, define its goal, duration and milestones, withdraw each milestone once approved.	Access funds before approval, change the milestones, or skip the order.
Backer	Donate while the campaign is open, vote on the current milestone, claim a refund if the goal is missed.	Vote on future milestones, vote twice on the same milestone, or withdraw after the goal is reached.
Anyone	Read all campaign data directly from the chain.	Alter any campaign.

5. Campaign Lifecycle

1. Creation

The creator calls `createCampaign` with a title, a funding goal in the chain's native token, a duration in days and parallel lists of milestone titles and percentages. The contract rejects the call if the goal or duration is zero, if the lists are empty or of different lengths, or if the percentages do not sum to 100. The deadline is set to the creation time plus the chosen number of days.

2. Fundraising

Until the deadline, anyone can call `donate` with a value. The amount is added to the campaign total and recorded against the donor's address. Donations are accepted even after the goal is reached, until the deadline passes.

3. Goal reached: voting opens

Once total raised is at least the goal, milestone voting becomes available. Backers vote on the current milestone only, which is the next one not yet released. Each backer's vote carries a weight equal to their total contribution, and each address can vote once per milestone.

4. Release

The creator can call `release` when approval for the current milestone exceeds half of the total raised. The contract computes the payout, updates its accounting and then sends the funds. The next milestone becomes the current one.

5. Failure and refunds

If the deadline passes and the goal was not reached, any backer can call `refund` to receive their full contribution. Refunds are all-or-nothing per backer and can be claimed once.

6. Governance and Payout Mathematics

Approval and payout are deterministic functions of the campaign state.

```
approved(m)  =  votes[m] * 2  >  raised
payout(m)    =  raised * percent[m] / 100          (all but last milestone)
payout(last) =  raised - alreadyPaid               (final milestone sweeps the remainder)
```

Sweeping the remainder on the last milestone guarantees that integer rounding never strands dust in the contract. A worked example follows.

Item	Value
Funding goal	5 units
Total raised	6 units (over-funded)
Milestones	Prototype 30%, Beta 40%, Launch 30%
Prototype payout	6 x 30 / 100 = 1.8 units
Beta payout	6 x 40 / 100 = 2.4 units
Launch payout	6 - 1.8 - 2.4 = 1.8 units

Because payouts are a percentage of the total raised, over-funding increases every milestone proportionally. Voting weight is by contribution, so a single backer holding more than half of the funds can approve alone. This is a deliberate simplicity trade-off discussed under risks.

7. Smart Contract Design

The protocol is a single contract, `MilestoneCrowdfund`. It holds an array of campaigns and three mappings for contributions, vote weights and voting records. It has no owner, no proxy and no administrative functions.

Function	Access	Purpose
createCampaign	Anyone	Create a campaign with goal, duration and milestones.
donate	Anyone, before deadline	Add funds to a campaign.
vote	Donors, when goal reached	Approve the current milestone with weight equal to contribution.
release	Creator only	Withdraw the current milestone when approval exceeds 50%.
refund	Donors, after failed deadline	Reclaim a full contribution.
getCampaign / getMilestone	View	Read campaign and milestone data for interfaces.
campaignCount	View	Number of campaigns created.

Events

The contract emits `Created`, `Donated`, `Voted`, `Released` and `Refunded` events, which allow indexers and dashboards to reconstruct the full history of any campaign without trusting a server.

Safety patterns

- State is updated before any native-token transfer is made, so a recipient contract cannot re-enter and withdraw twice.
- Transfers use a low-level call and the result is checked; a failed transfer reverts the whole transaction.
- Solidity 0.8 checked arithmetic is used throughout, so overflows revert.
- There is no upgrade mechanism. A bug cannot be patched in place, but equally the operator cannot change the rules for existing campaigns.

8. Application Layer

The reference web application is a static site with no backend. It is organised as a landing page, a campaign browser with status filters, a campaign detail page showing milestone-by-milestone approval progress, and a creation form with live validation that milestone percentages total 100. Wallet connection prompts users to switch to or add BOT Chain testnet. All reads come directly from the chain through the public RPC endpoint, so the interface can be hosted anywhere and replaced by anyone.

9. Deployment on BOT Chain

inTrust targets BOT Chain's EVM-compatible testnet. BOT Chain's low fees make the multiple small transactions that milestone funding requires (donations, votes, releases) practical for small campaigns.

Parameter	Value
Network	BOT Chain testnet
Chain ID	968
RPC endpoint	https://rpc.bohr.life
Block explorer	https://scan.bohr.life
Contract address	Published at deployment
Language / compiler	Solidity 0.8.24 (checked on solc 0.8.26)

10. Security Considerations and Limitations

inTrust minimises trusted parties but does not eliminate every risk. Participants should understand the following before using it.

- **Not audited.** The contract is a reference implementation and has not been independently reviewed.
- **Whale control.** Because votes are weighted by contribution, a backer holding more than half of the funds decides every milestone. Campaigns dominated by one backer are effectively controlled by that backer.

- **Stalled campaigns.** After the goal is reached there is no refund path. If the creator disappears or a milestone can never reach majority approval, the remaining funds stay locked. Version 1 trades this risk for simplicity.
- **Late donations dilute approval.** Donations arriving after votes are cast raise the total that approval is measured against, so a milestone that was close to approval may need further votes.
- **Approval is social, not verified.** The contract cannot check that work was delivered. Backers must evaluate evidence off-chain, and creators should publish proof alongside each milestone request.
- **Native token only.** Funds are denominated in the chain's native token and are exposed to its price volatility.

11. Roadmap

The following items are proposed directions for future versions.

Stage	Focus
Next	Independent audit; test suite with fuzzing of the vote and payout paths; verified source on the block explorer.
Then	Stablecoin (ERC-20) campaigns; per-milestone deadlines with a backer-initiated refund of unreleased funds if a milestone is not approved in time.
Later	Optional oracle or reviewer attestations as an additional approval signal; quadratic or capped voting to reduce whale control; campaign metadata and evidence links stored with each milestone.

12. Conclusion

inTrust replaces the question "do I trust this creator with my money?" with a narrower one: "do I believe this milestone was delivered?" By holding funds in escrow, tying payouts to backer approval and guaranteeing refunds for failed campaigns, it gives builders a credible way to ask for support and gives backers an enforceable say in how it is spent.

Disclaimer

> This document describes software that is experimental, deployed on a test network and has not been independently audited. Nothing here is financial, legal or investment advice, and nothing here promises returns or the availability of any feature. Test-network tokens have no monetary value. Roadmap items are intentions, not commitments.